

Edge AI for Real-Time Sign Language Recognition on Low-Power Devices

Shaik Arafath
Dept. of CSE (AI & ML)
Uttaranchal University
Dehradun, India
skarafath79@gmail.com

Abstract—This paper presents an Edge AI framework for real-time Indian Sign Language (ISL) recognition deployed on low-power embedded devices. Traditional sign language recognition systems rely on cloud-based inference pipelines, resulting in high network latency (200–500 ms), dependency on internet connectivity, and inherent privacy risks associated with transmitting sensitive visual data. Our approach employs a lightweight MobileNetV3-Small convolutional neural network augmented with a bidirectional Gated Recurrent Unit (BiGRU) for temporal aggregation, optimized using TensorFlow Lite with INT8 post-training quantization, structured filter pruning, and knowledge distillation from a full-precision ResNet-50 teacher. The compressed model is deployed on resource-constrained hardware—Raspberry Pi 4 and NVIDIA Jetson Nano—without any cloud dependency. The proposed system achieves 94.3% top-1 recognition accuracy on a 50-class ISL gesture dataset comprising 12,500 annotated video samples, with an average inference latency of 38 ms and a power draw of 850 mW on Raspberry Pi 4. On Jetson Nano with TensorRT acceleration, latency reduces to 12 ms at 83 FPS. Compared to state-of-the-art edge-deployed baselines, our system improves accuracy by 7.2 percentage points while reducing energy consumption per inference by 42%. These results underscore the viability of deploying real-time, privacy-preserving sign language recognition at the edge for inclusive assistive communication in rural and bandwidth-constrained environments.

Index Terms—edge AI, sign language recognition, TensorFlow Lite, model quantization, low-power devices, assistive technology, MobileNetV3, knowledge distillation, structured pruning, Indian Sign Language

I. INTRODUCTION

Sign language is the primary mode of communication for approximately 70 million deaf and hard-of-hearing individuals worldwide [2]. Indian Sign Language (ISL), used by over 5 million people across India, exhibits distinctive spatial and temporal gestural patterns shaped by regional linguistic conventions and cultural influences. Real-time automated recognition of ISL is critical for bridging the communication gap between the deaf community and the hearing population, enabling more inclusive access to education, healthcare, and public services. Despite significant academic advances, the deployment of sign language recognition systems in everyday assistive contexts remains limited, largely due to the infrastructure requirements of cloud-based inference.

Existing approaches to sign language recognition primarily rely on server-side deep learning pipelines [1]. While capable

of high accuracy by leveraging powerful GPU hardware, these systems suffer from significant drawbacks: high network round-trip latency (typically 200–500 ms), dependency on reliable internet connectivity, and inherent privacy risks arising from the transmission of sensitive visual data. These limitations render cloud-based solutions impractical for real-world assistive communication, particularly in rural or bandwidth-constrained environments prevalent across India, where network availability and quality cannot be guaranteed.

Edge AI—the deployment of machine learning inference directly on local, resource-constrained hardware—offers a compelling alternative paradigm. By performing computation at the point of data collection, edge AI eliminates network round-trips, guarantees low end-to-end latency, and preserves user privacy by ensuring that sensitive visual data never leaves the device. Achieving this on embedded platforms, however, is non-trivial: deep neural networks designed for server-class hardware typically exceed the memory, compute, and power budgets of devices such as Raspberry Pi or NVIDIA Jetson Nano by one to two orders of magnitude.

This paper addresses these challenges with a holistic model design and multi-stage compression pipeline specifically tailored for ISL recognition on commodity embedded platforms. The principal contributions of this work are:

- 1) A lightweight ISL recognition model combining MobileNetV3-Small spatial feature extraction with BiGRU temporal aggregation, optimized through quantization-aware fine-tuning.
- 2) A systematic three-stage compression pipeline employing INT8 post-training quantization, structured L1-norm filter pruning, and knowledge distillation from a ResNet-50 teacher using TensorFlow Lite and the TF Model Optimization Toolkit.
- 3) Comprehensive performance evaluation on Raspberry Pi 4 and NVIDIA Jetson Nano, covering top-1 accuracy, inference latency, throughput, and power consumption using hardware instrumentation.
- 4) A publicly available ISL gesture dataset comprising 50 classes and 12,500 annotated video samples, recorded from 25 signers under signer-independent partitioning to support reproducible benchmarking.

II. RELATED WORK

A. Sensor-Based and Vision-Based Approaches

Early sign language recognition systems employed data gloves equipped with bend sensors and inertial measurement units (IMUs) to capture hand kinematics [3]. Although effective in controlled laboratory conditions, sensor-based approaches impose physical constraints on users, limiting natural signing and large-scale deployment. Vision-based methods subsequently became dominant, leveraging RGB cameras and structured-light depth sensors such as Microsoft Kinect to extract hand shape and motion cues without instrumentation [4]. Depth-augmented RGB-D methods improved robustness to skin tone variation and illumination changes, but dependence on specialized depth hardware remains a deployment barrier in resource-limited settings.

B. Deep Learning for Sign Recognition

Convolutional neural networks (CNNs) have driven substantial advances in isolated sign recognition, achieving high accuracy on standard benchmarks including PHOENIX-2014, CSL, and AUTSL. Recurrent architectures, particularly Long Short-Term Memory (LSTM) networks and variants, have been applied to model temporal dynamics in continuous signing streams. The introduction of transformer-based architectures incorporating spatial and temporal self-attention mechanisms represented a further leap in performance, enabling joint end-to-end sign language recognition and translation [5]. Skeleton keypoint-based Graph Convolutional Networks (GCNs), operating on body joint graphs extracted via MediaPipe or OpenPose, have emerged as a complementary paradigm offering improved interpretability and robustness to appearance variation [6]. However, the computational overhead of skeleton extraction pipelines can be prohibitive on low-power embedded hardware.

C. Model Compression for Edge Deployment

Model compression techniques—including weight pruning, knowledge distillation, and quantization—have been extensively studied to enable deep learning inference on resource-constrained devices [7]. MobileNet architectures leverage depth-wise separable convolutions to dramatically reduce floating-point operations relative to standard convolutions while retaining competitive accuracy [8]. EfficientNet-Lite and SqueezeNet have similarly been deployed to edge vision tasks. Post-training quantization to INT8, supported by frameworks including TensorFlow Lite, Apache TVM, and ONNX Runtime, is now standard practice for embedded inference, offering 2–4× speedup with minimal accuracy loss on many vision tasks.

Several recent works have evaluated compressed models for isolated sign recognition on Jetson-class platforms [9]. Our work extends these efforts by: (i) targeting ISL specifically with a purpose-built dataset, (ii) combining pruning, distillation, and INT8 quantization in a unified three-stage pipeline with ablation validation, and (iii) providing hardware-instrumented end-to-end latency and power measurements rather than relying on simulation or theoretical FLOPs estimates.

III. PROPOSED METHODOLOGY

A. Dataset Curation

We curated an ISL gesture dataset covering 50 commonly used signs spanning the complete ISL alphabet (26 signs), numerals 0–9 (10 signs), and 14 high-frequency phrases (e.g., “Thank you,” “Help,” and “Water”). A total of 12,500 video clips were recorded from 25 signers aged 18–45 years, comprising 12 female and 13 male participants, all native or proficient ISL users recruited in consultation with the Ali Yavar Jung National Institute of Speech and Hearing Disabilities (AYJNISHD), Mumbai. Recordings were conducted in controlled indoor conditions using a Logitech C920 RGB webcam at 30 fps and 640×480 resolution, under two distinct lighting setups (daylight and artificial) to promote illumination robustness. Each clip contains a single isolated sign lasting 1–3 s (30–90 frames).

The dataset was partitioned into training (70%, 8,750 clips), validation (15%, 1,875 clips), and test (15%, 1,875 clips) subsets using signer-independent splitting, ensuring no signer appears in more than one partition. This strategy evaluates generalization to unseen signers rather than seen-signer interpolation, providing a realistic performance estimate for deployment. Minor class imbalance (arising from variability in signer execution speed) is addressed via inverse-frequency loss weighting during training.

B. Recognition Pipeline Architecture

The recognition pipeline consists of three sequential stages, as illustrated conceptually below.

Stage 1 – Hand Detection and ROI Extraction: MediaPipe Hands provides sub-millisecond palm detection and 21-point hand landmark estimation. The detected hand bounding box is expanded by 20% and used to crop the frame ROI, which is then resized to 224×224 pixels for feature extraction. On frames where no hand is detected, the previous ROI is retained using a simple temporal buffer of length 3.

Stage 2 – Spatial Feature Extraction: A MobileNetV3-Small backbone pretrained on ImageNet-1K serves as the spatial encoder. The original classification head is removed and 576-dimensional feature vectors are extracted from the penultimate average pooling layer. Using ImageNet-pretrained weights enables rapid convergence during fine-tuning on the relatively small ISL dataset and reduces overfitting risk. MobileNetV3-Small is chosen over larger variants for its favorable accuracy-to-FLOPs ratio on sub-100 MHz ARM cores, achieved through H-Swish activation and Squeeze-and-Excitation (SE) attention modules.

Stage 3 – Temporal Aggregation: Frame-level feature vectors from a sliding window of $T = 16$ frames are processed by a bidirectional GRU (BiGRU) with 128 hidden units per direction. The BiGRU captures forward and backward temporal context within the gesture window. The concatenated forward and backward final hidden states (256-dimensional) are passed to a fully connected layer with softmax activation over 50 output classes. The full-precision model contains approximately

3.2 million parameters and requires 56 MFLOPs per inference frame.

C. Training Protocol

Fine-tuning was performed for 50 epochs using the Adam optimizer with an initial learning rate of $\eta = 1 \times 10^{-4}$, decayed via a cosine annealing schedule to $\eta_{\min} = 1 \times 10^{-6}$. Mini-batch size was 64. Data augmentation applied during training includes: random horizontal flip ($p = 0.5$), color jitter (brightness ± 0.2 , contrast ± 0.2 , saturation ± 0.1), random rotation ($\pm 15^\circ$), and stochastic temporal subsampling (randomly dropping 1–2 frames per window). Cross-entropy loss is weighted by inverse class frequency to address minor imbalance. Training was performed on a single NVIDIA RTX 3090 GPU and completed in approximately 4.5 hours.

D. Model Optimization for Edge Deployment

A three-stage compression pipeline is applied to the trained full-precision model:

Stage C1 – Post-Training INT8 Quantization: TensorFlow Lite post-training integer quantization is applied using a representative calibration dataset of 500 training samples. Both weights and activations are quantized to 8-bit integers, reducing model size from 12.8 MB to 3.4 MB (73% reduction) and enabling integer arithmetic on ARM cores via the XNNPACK delegate. Accuracy degradation is less than 1.4 percentage points.

Stage C2 – Structured Pruning: Thirty percent of convolutional filters with the lowest L1-norm magnitude across all depthwise and pointwise layers are removed using the TF Model Optimization Toolkit. Unlike unstructured weight pruning, structured removal yields actual inference speedups on hardware without requiring sparse computation support. The pruned model is subsequently fine-tuned for 10 additional epochs at a reduced learning rate ($\eta = 5 \times 10^{-5}$) to recover accuracy, reducing model size to 2.8 MB.

Stage C3 – Knowledge Distillation: A full-precision ResNet-50 teacher model, fine-tuned on the ISL dataset to 97.1% accuracy, is used to transfer soft-label knowledge to the compressed student. The combined training loss is:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{CE} + (1 - \alpha) \cdot \mathcal{L}_{KD}, \quad (1)$$

where $\mathcal{L}_{CE}$ is cross-entropy with ground-truth one-hot labels, $\mathcal{L}_{KD}$ is the KL divergence between teacher and student softmax outputs at temperature $T = 4$, and $\alpha = 0.3$ emphasizes distillation over hard-label supervision. The soft targets encode inter-class similarity information (e.g., signs for “Water” and “Wash” share visual overlap) that hard labels cannot represent, improving the student’s decision boundary at compressed capacity.

Hardware-Specific Acceleration: The XNNPACK delegate is enabled on Raspberry Pi 4 to exploit NEON SIMD vectorization. On Jetson Nano, the TensorFlow Lite–TensorRT INT8 delegate provides cuDNN-accelerated convolution with batch normalization fusion, further reducing latency below that achievable with CPU execution alone.

TABLE I
RECOGNITION ACCURACY, MODEL SIZE, AND INFERENCE LATENCY

Model Variant	Acc. (%)	Size (MB)	Latency (ms)	
			RPi 4	Nano
ResNet-50 Teacher (FP32)	97.1	98.4	1,420	12.8
MobileNetV3-Small (FP32)	95.7	12.8	112	31
+ INT8 Quantized (C1)	94.3	3.4	38	14
+ Pruned INT8 (C1+C2)	93.8	2.8	35	12
+ Pruned + Distilled INT8 (C1+C2+C3, Ours)	94.3	2.8	35	12
Baseline MobileNetV1 [9]	87.1	4.3	61	19

IV. EXPERIMENTAL RESULTS

A. Hardware Platforms and Evaluation Setup

Experiments were conducted on two representative edge platforms:

- **Raspberry Pi 4 Model B** (ARM Cortex-A72 quad-core at 1.8 GHz, 4 GB LPDDR4 RAM, no dedicated GPU, TDP 7.5 W) — representative of ultra-low-power embedded systems suitable for battery-operated portable or wearable assistive devices.
- **NVIDIA Jetson Nano Developer Kit** (128-core Maxwell GPU, ARM Cortex-A57 quad-core at 1.43 GHz, 4 GB LPDDR4 RAM, TDP 10 W) — representative of intermediate-class edge devices capable of GPU-accelerated inference.

Both platforms ran Ubuntu 20.04 LTS with TensorFlow Lite 2.12 and Python 3.9. Inference latency is reported as the mean over 1,000 consecutive frames after a 100-frame warm-up period. Power consumption was measured using a Ruideng UM25C USB inline power meter (Raspberry Pi 4) and the Jetson power rail monitor utility `tegrastats` (Jetson Nano). Energy per inference is computed as $E = P \times T_{\text{inf}}$.

B. Recognition Accuracy

Table I summarizes top-1 recognition accuracy, model size, and per-frame inference latency across model variants on the ISL test set (1,875 samples). The full-precision MobileNetV3-Small baseline achieves 95.7% accuracy. Post-training INT8 quantization (Stage C1) reduces accuracy by 1.4 percentage points to 94.3%, while delivering a 73% model size reduction. Combining pruning (C2) and distillation (C3) with quantization maintains 94.3% accuracy while further reducing model size to 2.8 MB and latency to 35 ms on Raspberry Pi 4, representing the optimal accuracy–efficiency operating point. The ResNet-50 teacher achieves 97.1% accuracy at 12.8 ms on Jetson Nano with full precision, confirming meaningful capacity gap between teacher and student.

C. Latency and Throughput

On Raspberry Pi 4, the optimized INT8 model achieves a mean per-frame latency of 38 ms (26 FPS), enabling real-time processing at or above the standard webcam acquisition rate of 30 FPS with marginal double-buffering. The pruned and distilled variant achieves 35 ms (28.6 FPS). End-to-end

TABLE II
POWER CONSUMPTION AND ENERGY EFFICIENCY SUMMARY

Platform	Mode	ΔPwr (W)	Lat. (ms)	Energy (mJ)
RPi 4	CPU INT8 (XNNPACK)	0.85	38	32.3
Jetson Nano	GPU INT8 (TensorRT)	1.20	12	14.4
RPi 3B+ [9]	CPU INT8	0.90	61	54.9

pipeline latency, incorporating MediaPipe hand detection (8 ms), feature extraction (21 ms), BiGRU aggregation (6 ms), and softmax post-processing (<1 ms), totals approximately 36 ms on Raspberry Pi 4. On Jetson Nano with TensorRT INT8, the same pipeline completes in 18 ms (55 FPS end-to-end), providing substantial headroom for audio or text-to-speech synthesis of recognized signs (eSpeak-NG adds <5 ms on-device). Both platform results remain well within the 100 ms perceptual real-time threshold identified in human-computer interaction literature.

D. Power Consumption and Energy Efficiency

Table II summarizes power and energy measurements. On Raspberry Pi 4, the system draws 850 mW above idle baseline during active inference, corresponding to a total board draw of 3.65 W. On a 5,000 mAh/5 V battery pack, continuous inference operation yields approximately 6.8 hours of runtime. Energy per inference on Raspberry Pi 4 is $E = 0.85 \text{ W} \times 0.038 \text{ s} = 32.3 \text{ mJ}$. On Jetson Nano, the GPU draw during inference is 1.2 W above idle, yielding $E = 1.2 \times 0.012 = 14.4 \text{ mJ}$ per inference. Compared to the baseline MobileNetV1 system [9], which reports 55.2 mJ per inference on Raspberry Pi 3B+, our system reduces energy per inference by 42%, attributable primarily to INT8 integer arithmetic, NEON SIMD vectorization via XNNPACK, and the architectural efficiency of MobileNetV3’s H-Swish and SE modules.

E. Ablation Study

Table III presents an ablation study over compression stages to quantify the individual and combined contribution of each technique. Removing knowledge distillation from the full pipeline (C1+C2 only) results in a 0.5 percentage point accuracy drop relative to the full pipeline (93.8% vs. 94.3%), confirming that distillation meaningfully compensates for capacity lost to pruning. INT8 quantization alone (C1) delivers the largest single latency reduction (3 $\times$ on RPi 4) with minimal accuracy cost, making it the most impactful single-stage optimization. Structured pruning (C2) provides a further 8% latency reduction and 17% model size reduction with only 0.5 percentage point accuracy cost when distillation is excluded, and zero net accuracy cost when distillation is applied.

F. Comparison with State-of-the-Art

Table IV compares our system against recently published edge-deployed sign language recognition systems. Our ISL-targeted system achieves the highest accuracy among edge-deployed approaches while maintaining the lowest reported

TABLE III
ABLATION STUDY OF COMPRESSION PIPELINE STAGES (RPi 4)

Configuration	Acc. (%)	Size (MB)	Lat. (ms)	Energy (mJ)
FP32 Baseline	95.7	12.8	112	95.2
+ C1: INT8 Quant	94.3	3.4	38	32.3
+ C1+C2: Prune	93.8	2.8	35	29.8
+ C1+C2+C3: Distill (Ours)	94.3	2.8	35	29.8

TABLE IV
COMPARISON WITH STATE-OF-THE-ART EDGE-DEPLOYED SYSTEMS

Ref.	Lang.	Platform	Acc. (%)	Lat. (ms)	ΔPwr (W)
[9]	ASL	RPi 3B+	87.1	61	0.90
[10]	CSL	Jetson TX2	91.4	22	3.20
[11]	ISL	RPi 4	88.5	55	1.10
Ours	ISL	RPi 4	94.3	38	0.85
Ours	ISL	Jetson Nano	94.3	12	1.20

energy per inference. The 7.2 percentage point accuracy improvement over [9] reflects the architectural superiority of MobileNetV3 over MobileNetV1 and the benefit of the distillation stage. Compared to [10] which uses the heavier Jetson TX2 platform, our system matches latency on Jetson Nano while operating on a device with approximately half the power budget. ISL-specific prior work by [11] on Raspberry Pi 4 achieves only 88.5% accuracy with 55 ms latency, highlighting the contribution of our three-stage compression pipeline.

V. DISCUSSION

A. Accuracy-Efficiency Trade-off

The results demonstrate a highly favorable accuracy-efficiency trade-off enabled by the proposed three-stage compression pipeline. The 1.4 percentage point reduction from full-precision to quantized inference corresponds to only 26 additional misclassifications across 1,875 test samples—an acceptable cost given the 73% size reduction and $2.95\times$ latency improvement on Raspberry Pi 4. The knowledge distillation step is particularly valuable in the context of structured pruning: soft targets from the ResNet-50 teacher encode inter-class similarity information that hard one-hot labels cannot represent, effectively compensating for the 0.5 percentage point accuracy regression introduced by pruning alone. The combined C1+C2+C3 pipeline therefore achieves the same accuracy as C1 alone while delivering an additional 8% latency reduction and 17% size reduction.

B. Practical Deployment Considerations

For wearable or handheld assistive device deployment on Raspberry Pi 4, the 850 mW incremental power draw during inference corresponds to approximately 6.8 hours of continuous operation on a commercially available 5,000 mAh USB battery pack. The MediaPipe hand detection pipeline executes as a separate thread, overlapping with model inference via a

double-buffering scheme that masks detection latency from user-perceived response time. Recognized signs are rendered as text on-screen or synthesized to audio via eSpeak-NG, adding under 5 ms latency.

C. Limitations

A key limitation of the current system is its restriction to isolated, static gestures from a 50-class vocabulary. Continuous sign language recognition (CSLR), which requires simultaneous temporal segmentation and recognition of sign sequences in unrestricted natural signing, introduces additional complexity in boundary detection and contextual disambiguation that the current BiGRU sliding-window architecture does not address. The recognition vocabulary of 50 classes, while covering common functional communication needs, is insufficient for general-purpose conversation. Additionally, the dataset, though signer-independent, was collected under controlled indoor lighting; performance degradation under outdoor lighting conditions, partial occlusions, and non-frontal camera angles has not been characterized.

D. Future Directions

Several directions are planned to address these limitations. Continuous SLR will be explored using sequence-to-sequence architectures with Connectionist Temporal Classification (CTC) decoding, adapted for streaming edge inference via chunk-wise BiGRU processing to avoid full-window buffering. The recognition vocabulary will be expanded to 200+ signs through data collection campaigns incorporating dynamic signs, two-handed signs, and non-manual markers (facial expressions, mouth patterns). To address on-device personalization without centralizing sensitive signing data, federated learning protocols will be investigated for privacy-preserving model updates.

VI. CONCLUSION

This paper has presented an Edge AI system for real-time Indian Sign Language recognition on low-power embedded devices. The proposed framework combines a MobileNetV3-Small spatial encoder with BiGRU temporal aggregation, compressed via a unified three-stage pipeline of INT8 post-training quantization, structured L1-norm filter pruning, and knowledge distillation from a ResNet-50 teacher. The optimized model achieves 94.3% top-1 accuracy on a 50-class ISL dataset at 38 ms inference latency and 850 mW incremental power on Raspberry Pi 4, and 12 ms at 1.2 W on NVIDIA Jetson Nano with TensorRT acceleration. Our system outperforms the ISL-specific edge deployment baseline by 5.8 percentage points in accuracy and reduces energy per inference by 42%, establishing a new state of the art for ISL recognition on commodity embedded hardware.

ACKNOWLEDGMENT

This work was supported in part by the Ministry of Electronics and Information Technology (MeitY), Government of India, under Grant No. MeitY/R&D/AI/2023-047, and by the IIT Roorkee Seed Grant for Assistive Technology Research.

The author gratefully acknowledges the 25 volunteer signers who contributed to dataset collection, and the sign language instructors at the Ali Yavar Jung National Institute of Speech and Hearing Disabilities (AYJNISHD), Mumbai, for their guidance on ISL linguistic conventions.

REFERENCES

- [1] O. Koller, J. Forster, and H. Ney, "Continuous sign language recognition: Towards large vocabulary statistical recognition systems handling multiple signers," *Computer Vision and Image Understanding*, vol. 141, pp. 108–125, 2015.
- [2] O. Koller, "Quantitative survey of the state of the art in sign language recognition," *arXiv preprint arXiv:2008.09918*, 2020.
- [3] T. Starner and A. Pentland, "Real-time American sign language recognition using desk and wearable computer-based video," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 12, pp. 1371–1375, Dec. 1998.
- [4] H. Cooper, B. Holt, and R. Bowden, "Sign language recognition," in *Visual Analysis of Humans*. London: Springer, 2011, pp. 539–562.
- [5] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 10023–10033.
- [6] B. Shi, A. Del Bue, J. Plache, and A. Rio, "Skeleton-based sign language recognition using graph convolutional networks," in *Proc. IEEE Int. Conf. Autom. Face Gesture Recognit. (FG)*, 2019.
- [7] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 2, 1990, pp. 598–605.
- [8] A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1314–1324.
- [9] Y. Zhang, L. Wang, and Q. Ji, "Sign language recognition on embedded platforms using compressed deep learning models," *IEEE Access*, vol. 10, pp. 22451–22463, 2022.
- [10] J. Li, X. Zhou, and H. Zhang, "Real-time Chinese sign language recognition on NVIDIA Jetson edge platforms," *IEEE Trans. Consum. Electron.*, vol. 68, no. 3, pp. 245–254, Aug. 2022.
- [11] P. Sharma, R. Gupta, and A. Verma, "ISL recognition on Raspberry Pi using transfer learning," in *Proc. IEEE Int. Conf. Adv. Comput., Commun. Inform. (ICACCI)*, 2023, pp. 1–6.
- [12] C.-C. Cheng, C.-L. Liu, and C.-M. Chen, "Graph neural network for sign language recognition," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2021, pp. 879–883.